

Student Mental Health Monitoring System

SE LAB Experiment-06

Software Project Management Plan (SPMP)

Guided by IEEE Std 1058

Submitted By:

Mokshi Shah
241071068

Vanshika Shah
241071069

April 10, 2026

Contents

1	Project Overview	2
2	Project Scope	4
3	Work Breakdown Structure (WBS)	6
3.1	WBS Work Package Detail Table	8
4	Effort Distribution Chart	10
5	Schedule Plan	11
6	Resource Plan	12
7	Cost Plan	13
8	Risk Management Plan	14
9	Quality Plan	15
10	Communication Plan	17
11	Closure Plan	18

1 PROJECT OVERVIEW

Project Name	Student Mental Health Monitoring System (SMHMS)
Project Type	Web-based intelligent monitoring platform
Standard	IEEE Std 1058 – Software Project Management Plan
Team	Mokshi Shah (241071068), Vanshika Shah (241071069)
Team Size	2 developers
Duration	≈ 50 weeks (≈ 11.5 months)
Effort	2,976.4 person-hours (UCP-based estimate)
Baseline Cost	Rs. 34,82,388 Recommended Budget: Rs. 42,00,000
Technology Stack	Python (Flask), HTML/CSS/JS, SQLite/PostgreSQL
Development Model	Agile (2-week sprints), 21 sprints across 42 weeks
Repository	GitHub (version control + CI/CD via GitHub Actions)

- Build a proactive, data-driven mental health monitoring system for academic institutions that reduces counselor workload through automated risk triaging.
- Integrate PHQ-9/GAD-7 screening, academic performance data (GPA, attendance), and behavioral signals for automated multi-tier risk classification (Low/Medium/High/Critical).
- Provide role-based dashboards for four user roles: students, counselors, administrators, and IT security officers – each with tailored views and permissions.
- Deliver all 10 use case groups (UC-01 through UC-29) within the estimated schedule (50 weeks) and approved budget (Rs. 42,00,000).
- Maintain $\geq 85\%$ risk classification accuracy (validated against counselor assessments in UAT) and ≤ 2 -second dashboard response time at 200 concurrent users.
- Ensure full GDPR and institutional data governance compliance with tamper-evident audit logging and AES-256 data encryption at rest.
- Achieve $\geq 70\%$ voluntary student adoption within the first academic semester post-deployment.

Assumptions:

- All students have valid institutional email accounts that can be used for authentication.
- At least one counselor responds to high/critical alerts within 48 hours of generation.
- Campus-wide internet connectivity is reliable with $\geq 99\%$ uptime during academic hours.
- Student informed consent is obtained and documented before any data collection begins.
- A dedicated institutional server (minimum 8-core CPU, 16 GB RAM) is available for production deployment.

Constraints:

- The system must run entirely within institutional IT infrastructure; no student data may be sent to third-party cloud providers (AWS, GCP, Azure, etc.).
- SIS API integration is read-only; no write operations to the Student Information System are permitted.
- All features must comply with GDPR Article 5 data minimization and IEEE 29148 requirements engineering standards.
- The system must support WCAG 2.1 Level AA accessibility standards for students with disabilities.

Dependencies:

- SIS API documentation and sandbox access must be provided by the institution by Week 3.
- Server infrastructure (hardware + OS) must be provisioned by the IT department by Week 40.
- PHQ-9 and GAD-7 licensing/permissions must be confirmed with publishers before Week 6.
- At least two counselors must be available for UAT sessions during Weeks 30–34.

2 PROJECT SCOPE

- **Student Module:** Secure login with institutional SSO/2FA, standardized screening forms (PHQ-9, GAD-7, PSS-10), personalized risk status dashboard with trend visualization, daily mood logging with emoji/slider interface, curated wellness resource library, and counseling session booking with calendar integration.
- **Analytics Module:** Automated multi-source data collection (screening scores, academic data via SIS API, attendance, mood logs), time-series pattern analysis, weighted multi-factor risk scoring algorithm, automated alert generation (email + in-app), and longitudinal trend tracking with visual reports.
- **Counselor Module:** At-risk student prioritized list with filter/sort controls, full student profile views (screening history, academic data, mood trends), session scheduling and clinical notes interface, alert acknowledgment and escalation workflow, and PDF report export for case documentation.
- **Administrator Module:** User account lifecycle management (create/suspend/delete), institution-wide trend analytics and aggregate dashboards, configurable risk threshold parameters, scheduled and on-demand CSV/PDF report exports, and role assignment controls.
- **IT Security Module:** RBAC policy management with fine-grained permissions, tamper-evident audit logging (all data access and modifications), GDPR compliance monitoring dashboard, configurable data retention policies, and automated compliance alerts.
- **Deployment:** Fully containerized Docker deployment with docker-compose orchestration, nginx reverse proxy with TLS 1.3 termination, automated database backup scripts, and comprehensive deployment and runbook documentation.

The following are explicitly excluded from this project and will not be built, supported, or maintained:

- Clinical diagnosis, prescription recommendations, or any form of medical decision support.
- Electronic Medical Record (EMR) or hospital system integration of any kind.
- Payment processing or billing functionality (e.g., paid counseling sessions).
- Native iOS or Android mobile applications (the web application is mobile-responsive only).
- AI/LLM-based chatbot or conversational therapy features.
- Integration with external third-party mental health platforms (e.g., BetterHelp, Talkspace).

#	Deliverable	Description	Due (Week)
D1	Software Requirements Spec.	IEEE 830 SRS document	4
D2	System Design Document	Architecture + DB schema + wireframes	8
D3	Working Software (Sprint demos)	Bi-weekly sprint deliverables	8–34
D4	Test Reports	Unit, integration, UAT, security	36
D5	Deployed System	Live system on institutional server	46
D6	Documentation Package	User manual, API docs, runbook	50

3 WORK BREAKDOWN STRUCTURE (WBS)

The WBS decomposes the SMHMS project into four levels following the structure shown in the WBS levels diagram (TeamGantt reference). Level 1 is the project goal; Level 2 contains major phases; Level 3 contains sub-phases/deliverables; Level 4 contains individual work packages with effort estimates. The WBS is structured to align with the 21 Agile sprints planned across the 50-week project duration.

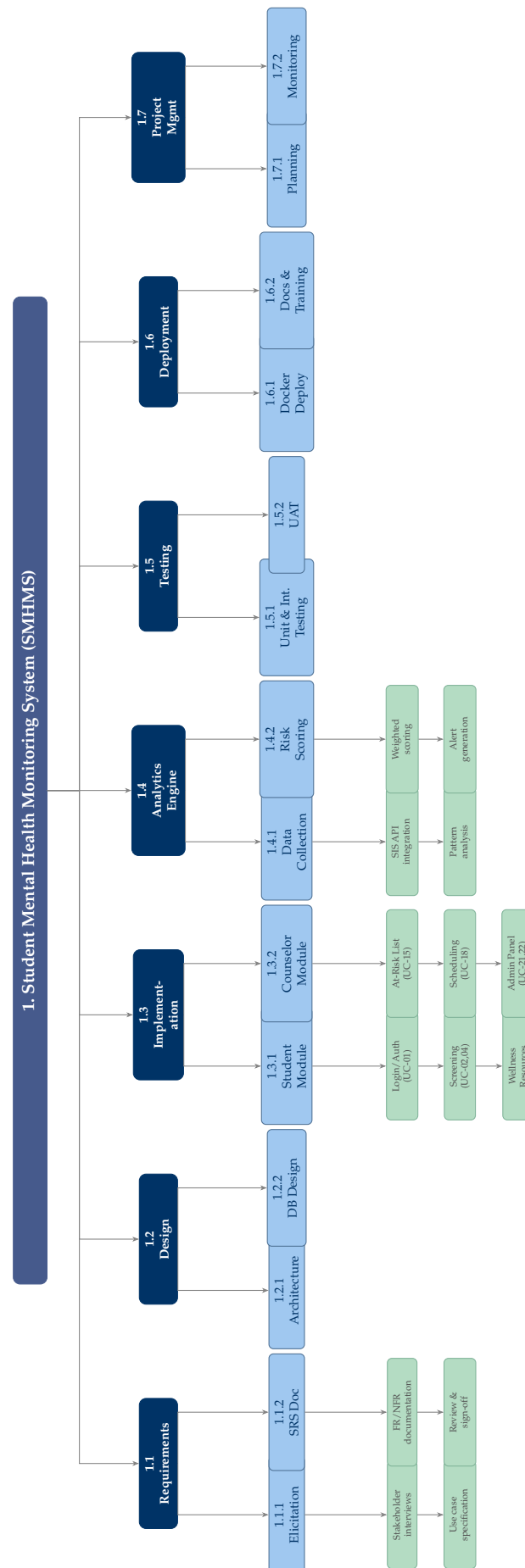


Figure 1: Work Breakdown Structure – SMHMS (Levels 1–4)

3.1 WBS Work Package Detail Table

WBS-ID	Work Package	Deliverable	Effort (hrs)	Weeks	Owner
1.1 Requirements Engineering					
1.1.1	Stakeholder elicitation	Use case list, interview notes	80	1–2	Both
1.1.2	SRS documentation	SRS document (IEEE 830)	100	2–4	Both
1.2 System Design					
1.2.1	System architecture	Architecture diagram, API spec	100	4–6	Both
1.2.2	Database design	ER diagram, schema scripts	80	5–7	Vanshika
1.2.3	UI/UX wireframes	Screen mockups (8 UI designs)	60	6–8	Mokshi
1.3 Implementation – Student & Counselor Modules					
1.3.1	Login & auth (UC-01)	Working login with 2FA	80	8–10	Vanshika
1.3.2	Screening forms (UC-02, 04)	PHQ-9/GAD-7 forms + dashboard	120	9–13	Mokshi
1.3.3	Wellness resources (FR-05)	Resource library page	40	12–14	Mokshi
1.3.4	Counselor at-risk list (UC-15)	Counselor dashboard	100	13–17	Both
1.3.5	Session scheduling (UC-18)	Calendar + booking interface	100	16–20	Vanshika
1.3.6	Admin & IT Security panel (UC-21,22,29)	Admin + security UIs	120	18–24	Both
1.4 Analytics Engine					
1.4.1	Data collection & SIS integration (FR-07)	ETL pipeline, SIS connector	120	10–16	Vanshika
1.4.2	Pattern analysis (UC-11)	Analysis algorithm module	140	14–20	Both
1.4.3	Risk scoring & alerts (UC-12, FR-10)	Scoring engine, alert service	120	18–24	Mokshi
1.5 Testing					
1.5.1	Unit & integration testing	Test reports, $\geq 80\%$ coverage	160	22–30	Both
1.5.2	User acceptance testing (UAT)	UAT sign-off document	80	30–34	Both
1.5.3	Security & performance testing	Pentest report, load test results	60	32–36	Vanshika
1.6 Deployment					
1.6.1	Docker containerization	Docker images, compose file	60	38–42	Mokshi
1.6.2	Institutional server deployment	Live system, deployment guide	60	40–46	Both
1.6.3	User documentation & training	User manuals, training sessions	40	46–50	Both

Note: Total from WBS = 1,924 hrs. UCP-based total = 2,976.4 hrs. The difference accounts for non-functional work (documentation overhead, stakeholder meetings, knowledge transfer, contingency buffer) included in the UCP estimate.

4 EFFORT DISTRIBUTION CHART

The charts below show the distribution of estimated effort (person-hours) across all seven major project phases. Values are derived from WBS work package estimates (total 1,930 hrs from WBS; 2,976.4 hrs from UCP-based model including overhead). Implementation dominates at 27% due to the breadth of five distinct role-based modules.

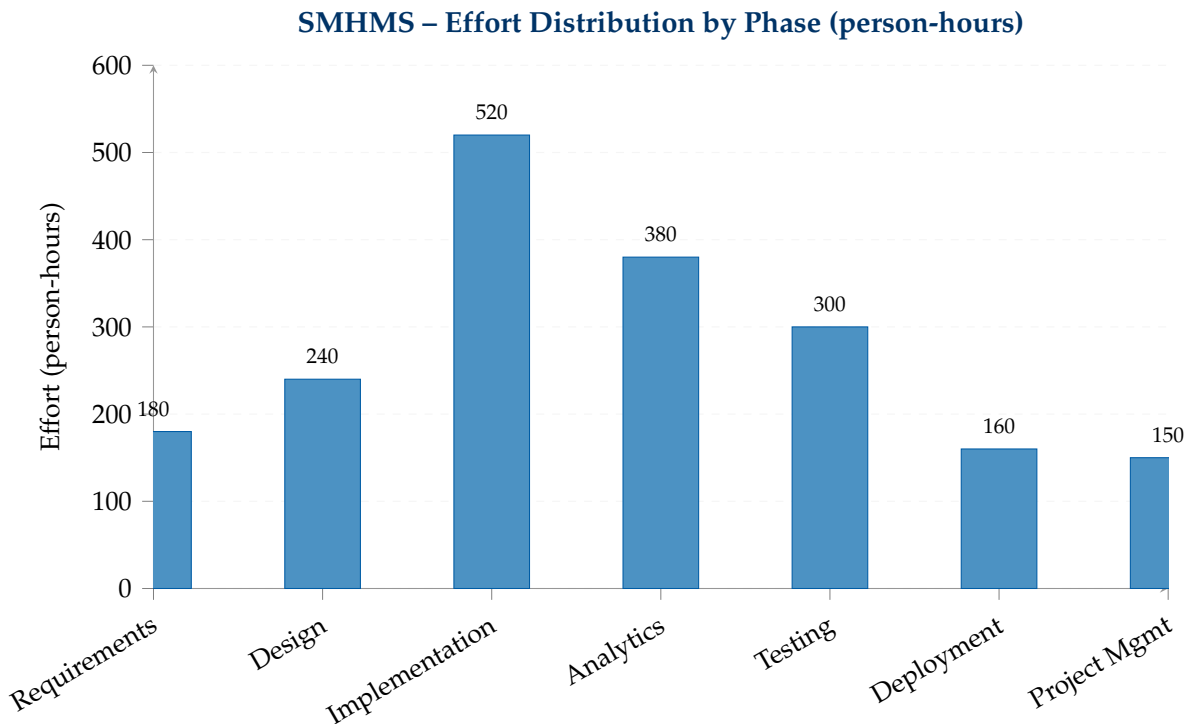


Figure 2: Effort Distribution across SMHMS Project Phases (WBS-based, 1,930 hrs total)

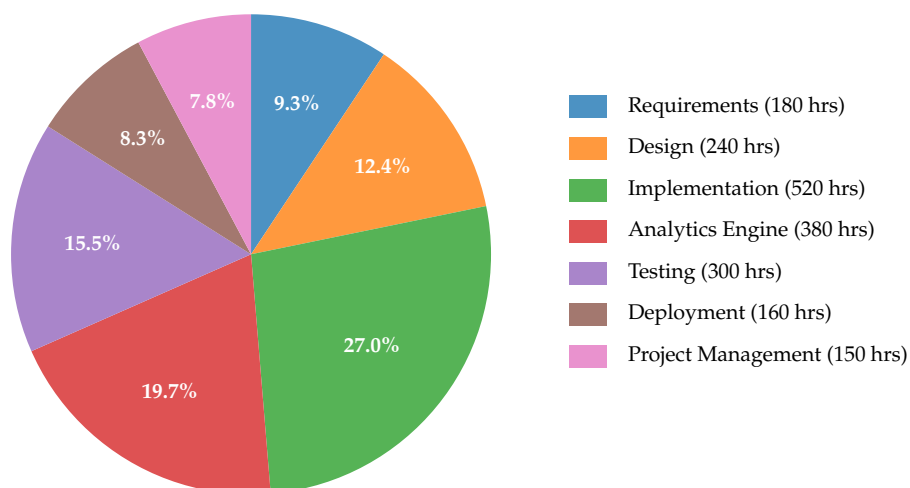


Figure 3: Effort Distribution Pie Chart – SMHMS (Total: 1,930 person-hours from WBS)

5 SCHEDULE PLAN

Phase	Activity	Start Wk	End Wk	Key Milestone / Deliverable
1	Requirements Engineering	1	4	SRS document approved by supervisor and counselor
2	System Design	4	8	Architecture, DB schema, UI wireframes reviewed and baselined
3	Implementation (Core)	8	20	Student, counselor, admin, IT security modules passing unit tests
4	Analytics Engine	10	24	Risk scoring engine operational; $\geq 85\%$ accuracy on test dataset
5	Integration	22	28	All modules integrated; end-to-end flow verified in staging
6	Testing	22	36	Unit, integration, UAT, security tests passed; $\geq 80\%$ coverage confirmed
7	Deployment	40	50	System live on institutional server; all documentation delivered

The project is executed in 21 two-week sprints. Key sprint goals are summarized below:

- **Sprints 1–2 (Wk 1–4):** SRS elicitation, use case documentation, project environment setup.
- **Sprints 3–4 (Wk 5–8):** System architecture, ER diagram, UI wireframes, DB schema.
- **Sprints 5–7 (Wk 9–14):** Student module – login/2FA, PHQ-9/GAD-7 forms, dashboard.
- **Sprints 8–10 (Wk 15–20):** Counselor module, session scheduling, SIS ETL pipeline.
- **Sprints 11–12 (Wk 21–24):** Admin/IT Security UI, analytics risk scoring engine.
- **Sprints 13–14 (Wk 25–28):** Full system integration, API contract testing.
- **Sprints 15–17 (Wk 29–34):** UAT with counselors, performance load testing.
- **Sprints 18–19 (Wk 35–38):** Security penetration testing, bug fixes, Docker packaging.
- **Sprints 20–21 (Wk 39–50):** Production deployment, documentation, training, handover.

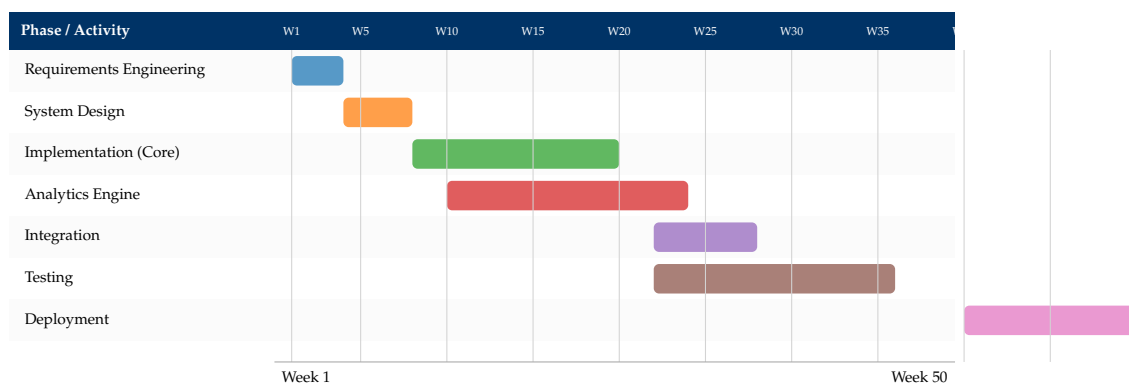


Figure 4: Project Schedule – Gantt Chart (Weeks 1–50, 7 Phases)

6 RESOURCE PLAN

Resource	Role	Allocation	Responsibilities
Mokshi Shah	Lead Developer / Analyst	100%	Frontend UI design, screening module (PHQ-9/GAD-7), analytics dashboard, report generation, documentation, sprint demos
Vanshika Shah	Lead Developer / Architect	100%	Backend REST API (Flask), PostgreSQL database design, SIS ETL integration, IT security module, Docker deployment, CI/CD pipeline
Institution Counselor	Domain Expert / UAT Lead	Part-time	Requirements validation for clinical features, co-design of risk threshold parameters, UAT execution and sign-off
IT Administrator	Infrastructure Support	Part-time	Server provisioning and OS configuration, network firewall rules, Docker deployment support, TLS certificate management
Academic Supervisor	Project Oversight	Bi-weekly	Sprint review attendance, SPMP approval, escalation point for project-level risks and scope decisions

Backend	Python 3.11, Flask 3.x, SQLAlchemy ORM, Celery (background task scheduling), Redis (task queue broker)
Frontend	HTML5, CSS3, JavaScript (Vanilla + Chart.js for data visualizations), Jinja2 templates, Bootstrap 5 (responsive layout)
Database	PostgreSQL 15 (production with connection pooling via PgBouncer), SQLite (local development)
Analytics	scikit-learn (logistic regression + random forest for risk classification), pandas (data processing and aggregation), numpy
Security	bcrypt (password hashing), PyJWT (token authentication), TLS 1.3 via nginx reverse proxy, AES-256 encryption for PII fields via SQLAlchemy-encrypted
DevOps	Docker 24+, Docker Compose v2, GitHub (version control), GitHub Actions (CI: lint + test + build pipeline)
Testing	pytest ($\geq 80\%$ line coverage), pytest-cov (coverage reporting), Selenium WebDriver (UAT automation), OWASP ZAP 2.14 (security scanning), Apache JMeter (load testing)
PM Tools	GitHub Projects (Kanban sprint board), GitHub Issues (bug tracking), Google Docs (shared documentation), Swagger/OpenAPI 3.0 (API documentation)

Component	Specification
Production Server	8-core CPU, 16 GB RAM, 500 GB SSD, Ubuntu 22.04 LTS
Development Machines	2× laptops with 8 GB RAM, Docker Desktop, Python 3.11
Network	HTTPS (port 443) via nginx; internal server on institutional LAN with firewall rules
Backup	Automated daily PostgreSQL pg_dump to encrypted backup storage; 30-day retention
Email (alerts)	Institutional SMTP relay for counselor alert notifications

7 COST PLAN

Cost Category	Calculation	Amount (Rs.)	Notes
Development Effort	2,976.4 hrs × Rs.900/hr	26,78,760	UCP-based estimate; Rs.900/hr is mid-level developer rate, India
Contingency Reserve (15%)	26,78,760 × 0.15	4,01,814	Risk buffer for scope changes, rework, or delays
Infrastructure / Tools (5%)	26,78,760 × 0.05	1,33,938	Server hardware, SSL certificates, licensed tools
Testing & QA (10%)	26,78,760 × 0.10	2,67,876	OWASP ZAP license, JMeter infrastructure, UAT facilitation
Total Project Cost		34,82,388	Baseline cost
Recommended Budget (20% buffer)		42,00,000	Approved budget target
Worst-Case Cost Cap		50,00,000	See sensitivity analysis below

Scenario	Effort Hrs	Total Cost (Rs.)	vs. Baseline
Optimistic (High TCF only, +21.4%)	3,612.0	32,50,800	+21.4%
Base Case (UCP estimate)	2,976.4	34,82,388	–
Low Experience (+24.4%)	3,703.6	33,33,240	+24.4%
Worst Case (all factors)	5,614.0	50,52,600	+88%

The

20% budget buffer (Rs. 42,00,000) covers the base case comfortably. Worst-case scenario (Rs. 50,00,000) triggers escalation to the academic supervisor and scope negotiation.

8 RISK MANAGEMENT PLAN

#	Risk	Prob.	Impact	Level	Mitigation Strategy
R1	Scope creep from new stakeholder requirements	Medium	High	High	Formal change control process; scope frozen after SRS sign-off (Week 4); all changes require supervisor approval
R2	SIS API unavailable or undocumented	Medium	High	High	Negotiate API access and obtain sandbox credentials by Week 3; design CSV import as a fully functional fallback
R3	Risk classification accuracy < 85%	Medium	High	High	Validate model with counselors during UAT (Wk 30–34); iteratively tune thresholds using F1-score metrics; fallback to rule-based scoring
R4	Team member unavailability (>1 week)	Low	High	Medium	Cross-train both members on all modules from Week 1; maintain detailed code comments and runbooks; 15% contingency in schedule
R5	Security vulnerability in student PII data	Low	High	Medium	OWASP ZAP automated scan + manual code review pre-deployment; AES-256 encryption at rest; TLS 1.3 in transit; RBAC with least-privilege
R6	Performance degradation under concurrent load	Low	Medium	Medium	JMeter load test to 500 concurrent users in Week 34; PgBouncer connection pooling; Celery async for heavy analytics tasks; Docker horizontal scaling ready
R7	Student adoption/engagement below 70%	Medium	Medium	Medium	Counselor-led orientation sessions; in-app nudge notifications for overdue screenings; UI/UX tested for ≤5 min onboarding time
R8	Schedule overrun due to analytics ML complexity	Medium	Medium	Low	Scope ML to established scikit-learn models only; allocate 4-week buffer in the analytics phase (Wk 10–24); monitor weekly in sprint reviews
R9	PHQ-9/GAD-7 licensing issues or delays	Low	Medium	Low	Initiate licensing request in Week 1 alongside SRS elicitation; GAD-7 is public domain; PHQ-9 freely available for non-commercial academic use
R10	GDPR/compliance breach during development	Low	High	Medium	Conduct data privacy review at end of each phase; IT Security officer reviews audit logs; no real student data used in dev/test environments

Avoid	R1 (scope freeze), R2 (early API negotiation), R9 (license obtained before Week 6).
Mitigate	R3 (model tuning), R5 (security controls), R6 (load testing), R10 (privacy reviews).
Transfer	Infrastructure risks transferred to the institutional IT department via SLA.
Accept	R4 (low probability), R7 (within operational control post-deployment), R8 (buffered in schedule).

9 QUALITY PLAN

Code Quality	All code peer-reviewed before merge to main branch; PEP-8 compliance enforced via flake8 linter in CI; $\geq 80\%$ statement coverage via pytest-cov.
Risk Accuracy	$\geq 85\%$ classification accuracy (precision and recall balanced) validated against counselor clinical assessment during UAT with ≥ 50 student records.
Performance	Dashboard page load ≤ 2 sec at 200 concurrent users; screening form submission ≤ 3 sec; verified via Apache JMeter 30-minute soak test.
Security	Zero critical or high CVSS-scored vulnerabilities before go-live (verified via OWASP ZAP automated scan + manual review of authentication and authorization flows).
Usability	New student completes first PHQ-9 screening in ≤ 5 minutes in unguided UAT session with $\geq 90\%$ task completion rate.
Accessibility	WCAG 2.1 Level AA compliance verified via axe-core automated checker plus manual keyboard-navigation testing.
Documentation	All API endpoints documented via Swagger/OpenAPI 3.0; user manual covers all four roles; deployment runbook covers end-to-end server setup.
Data Integrity	All PII fields encrypted at rest (AES-256); audit log entries immutable (append-only table); daily automated DB backups verified via restore test.

- **Sprint reviews (bi-weekly, Weeks 2–34):** Working software demo against sprint acceptance criteria; attended by both developers and academic supervisor.
- **Code reviews (every PR):** Peer review of every pull request before merge to main; checklist includes logic correctness, security anti-patterns, and test coverage.
- **Unit testing (continuous):** Developer-written pytest tests for every function; run automatically via GitHub Actions on every push.
- **Integration testing (post-module):** End-to-end API flow tests executed after each module is integrated into the shared staging environment.
- **UAT (Weeks 30–34):** Two counselors and five students test all 29 use cases against SRS acceptance criteria; defects logged in GitHub Issues with severity classification.
- **Penetration testing (Weeks 32–36):** OWASP ZAP automated scan targeting authentication endpoints, session management, input validation, and IDOR vulnerabilities.
- **Performance testing (Week 34):** JMeter load test simulating 500 concurrent users with 30-minute soak; ramp-up: 10 users/second; thresholds: ≤ 2 sec p95 response time.
- **Accessibility testing (Week 35):** axe-core automated scan + manual screen-reader test on student-facing pages to ensure WCAG 2.1 AA compliance.

Defects discovered during any testing phase are tracked via GitHub Issues with the following priority classification:

Priority	Definition	Example	Resolution SLA
Critical (P0)	System unusable or data loss	Auth bypass, DB corruption	Fix within 24 hrs
High (P1)	Major feature broken	Risk score not calculated	Fix within 72 hrs
Medium (P2)	Feature degraded	UI misalignment	Fix in next sprint
Low (P3)	Minor cosmetic issue	Typo, color mismatch	Fix if time permits

10 COMMUNICATION PLAN

Communication	Participants	Frequency	Medium	Purpose
Daily stand-up	Development team	Daily	WhatsApp / Google Meet	Sync on yesterday's progress, today's plan, and any blockers
Sprint planning	Development team	Bi-weekly	GitHub Projects	Backlog grooming, story point estimation, and task assignment
Sprint review/demo	Team + supervisor	Bi-weekly	Google Meet	Demo working software; gather feedback; adjust backlog
Stakeholder update	Team + counselors + IT admin	Monthly	Email + Meet	Requirements clarification, UAT scheduling, infrastructure updates
Risk review	Development team	Monthly	GitHub Issues	Update risk register; reassess probability/impact ratings
Defect triage	Development team	Weekly	GitHub Issues	Review and prioritize open bugs; assign owners
Final presentation	All stakeholders	End of project	In-person / hybrid	Project handover, live system demo, lessons-learned summary

- All project documents (SRS, SPMP, test reports, meeting minutes) are maintained in a shared Google Drive folder with version numbering (e.g., SPMP_v1.2.pdf).
- Source code and technical documentation are version-controlled in the GitHub repository with branching strategy: `main` (production-ready) ← `develop` (integration) ← `feature/*` (individual features).
- All documents are subject to review and approval by the academic supervisor before baseline is set; changes after baseline require a formal change request logged in GitHub Issues.

11 CLOSURE PLAN

The project is considered complete when all of the following conditions are satisfied:

- All 29 use cases (UC-01 through UC-29) implemented, tested, and UAT sign-off obtained from at least two counselors.
- System successfully deployed on the institutional production server and verified accessible to all four user roles (student, counselor, administrator, IT security officer).
- Test coverage $\geq 80\%$ (pytest-cov report attached); zero critical or high CVSS-scored security vulnerabilities outstanding.
- All documentation delivered and version-baselined: SRS (IEEE 830), SPMP (IEEE 1058), API docs (Swagger), user manuals for each role, and deployment runbook.
- Final project cost within the approved budget of Rs. 42,00,000 (actual cost report submitted).
- Post-deployment monitoring active: error logging (Flask logging + Sentry), uptime monitoring, and daily DB backup verified.
- Lessons-learned document completed and submitted to the academic supervisor.
- Knowledge transfer session conducted with the institutional IT administrator covering deployment, backup/restore, and common troubleshooting procedures.

Warranty Period	4 weeks of post-go-live support (Weeks 51–54) during which critical and high defects will be fixed at no additional cost.
Monitoring	Application error logs reviewed weekly; automated uptime alerts via email if system is unreachable for >5 minutes.
Handover	Full source code, Docker images, DB schema, and documentation handed to the institution’s IT team at project close.
Future Enhancements	Planned for Phase 2 (out of scope for this contract): native mobile app, LLM-based chatbot support, multi-institution deployment.